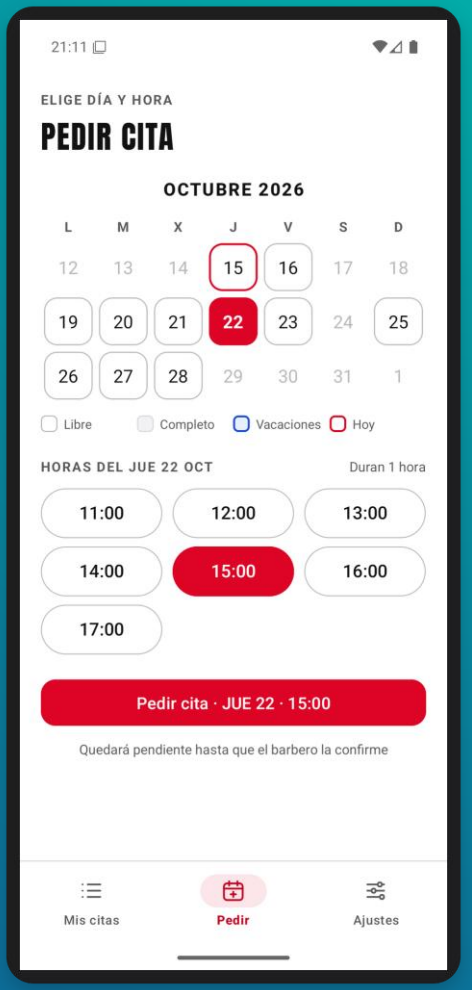
CLANDESTINO

App Android de citas para barberos que trabajan desde casa

Jhonnatan Ospina Henao
Tutor: Alfonso García
Curso 2025-2026
CFGS Desarrollo de Aplicaciones Multiplataforma

UNIVERSAE



1. Introducción al tema

Una app para el barbero que trabaja en casa

Desde casa

Sin local a pie de calle y con una clientela propia y fiel.

Clientes por invitación

Solo entra quien recibe un código y el barbero aprueba.

Cupo diario

Cada día admite un número máximo de citas.

La hice para mí. Ya uso una versión web propia y aquí rehago su núcleo como app Android nativa.

96,3 %

de las personas de 16 a 74 años usó Internet en los últimos tres meses

INE, 2025

94,6 %

de los internautas usa WhatsApp de forma habitual

CNMC, 2026

60,2 %

de las empresas de servicios personales no tiene asalariados

INE, DIRCE

1. Introducción al tema

Análisis de mercado: siete apps y ninguna cumple las tres reglas

App	Precio	Por qué no encaja
Booksy	34,99 € + IVA al mes y comisiones	Marketplace que enseña otras barberías
Fresha	19,95 € al mes y tarifa por cliente nuevo	Pensada para captar clientes
Treatwell	25 % de la primera reserva de cada cliente nuevo	Su negocio es traer clientes nuevos
AgendaPro	Desde 10 € al mes	Personal, inventario y caja
Square	Gratis, con comisión por cobro	Vive de su pasarela de pago
Setmore	Gratis hasta 4 usuarios, Pro a 5 $	Generalista, sin aprobación de clientes
Calendly	Gratis con un evento, 10 $ por usuario	Sin cupo diario ni aprobaciones

Fuente: webs oficiales de cada aplicación, consultadas el 30-09-2026.

CLANDESTINO

- ✓ Acceso solo por invitación y aprobación
- ✓ Cupo diario configurable
- ✓ Sin cuotas ni comisiones

A cambio, no sirve para salones con varios empleados ni cobra desde la app.

2. Objetivos planteados

Un objetivo general y cinco específicos

OBJETIVO GENERAL

Desarrollar una app Android, con su API y su base de datos, para que un barbero que trabaja desde casa gestione las citas de sus clientes con acceso por invitación, cupo diario y sin reservas duplicadas.

1. Base de datos

MySQL que impide por sí misma dos citas a la misma hora

2. API REST

Java sin frameworks, con contraseñas protegidas, tokens y roles

3. Cupo seguro

Se respeta aunque varios clientes pidan a la vez, y una prueba con hilos lo demuestra

4. App Android

MVVM, siete pantallas y la red fuera del hilo principal

5. Pruebas

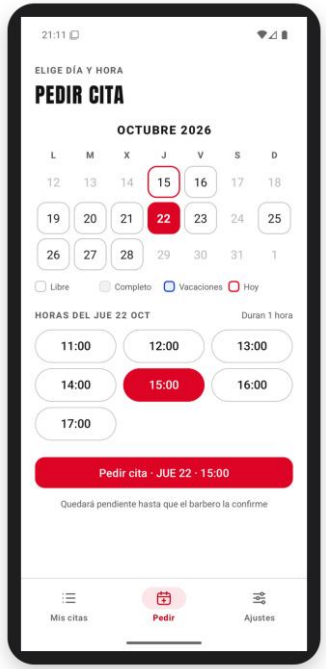
Unitarias, de integración y un recorrido completo en el emulador

3. Diseño y funcionalidad

Mockups: del dibujo a la pantalla real



Mockup en draw.io



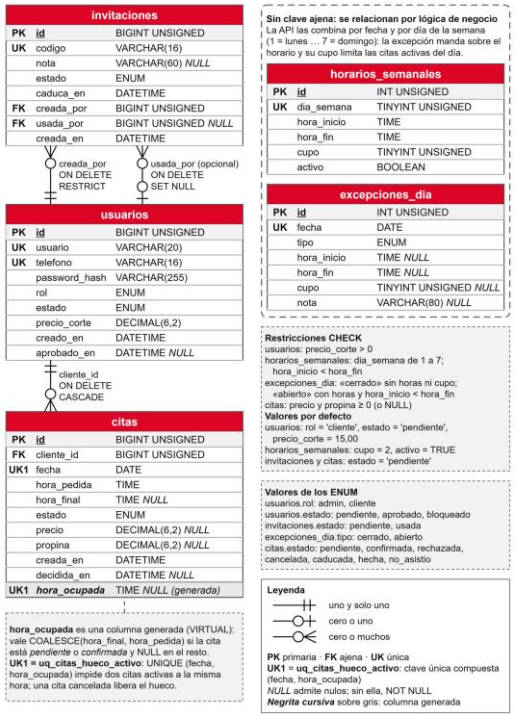
App en el emulador

11 mockups dibujados antes de escribir ningún layout, uno por pantalla y variante

- Identidad de mi marca: rojo para la acción principal y Anton en los títulos
- Componentes de Material 3, con zonas táctiles de 48 dp y tema claro y oscuro
- Al construirla, el calendario se quedó en las dos semanas reservables, sin flechas
- La agenda del barbero se quedó en vista semanal

3. Diseño y funcionalidad

Modelo de datos: cinco tablas contra la doble reserva



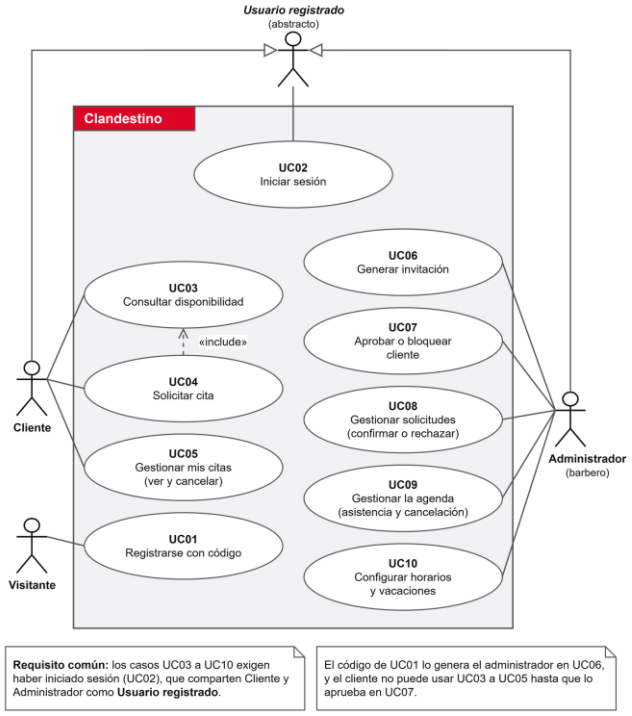
usuarios | **invitaciones** | **horarios_semanales** | **excepciones_dia** | **citas**

```
hora_ocupada TIME GENERATED ALWAYS AS (
  IF(estado IN ('pendiente','confirmada'),
     COALESCE(hora_final, hora_pedida), NULL))
UNIQUE KEY uq_citas_hueco_activo (fecha, hora_ocupada)
```

- Una cita cancelada o rechazada deja la columna a NULL, y el índice único admite varios NULL.
- Una segunda cita activa a la misma hora choca con el índice (error 1062) y la API responde 409 hueco_ocupado.
- El precio se copia en la cita al marcarla como hecha, así un cambio de tarifa no altera el historial.

3. Diseño y funcionalidad

Casos de uso: diez casos, todos programados



Visitante

UC01 Registrarse con código

Cliente

UC03 Consultar disponibilidad · UC04 Solicitar cita · UC05 Gestionar mis citas

Barbero

UC06 Generar invitación · UC07 Aprobar o bloquear · UC08 Solicitudes · UC09 Agenda · UC10 Horarios

UC02 Iniciar sesión es común a cliente y barbero.

30 requisitos funcionales **53** no funcionales

4. Metodología y planificación

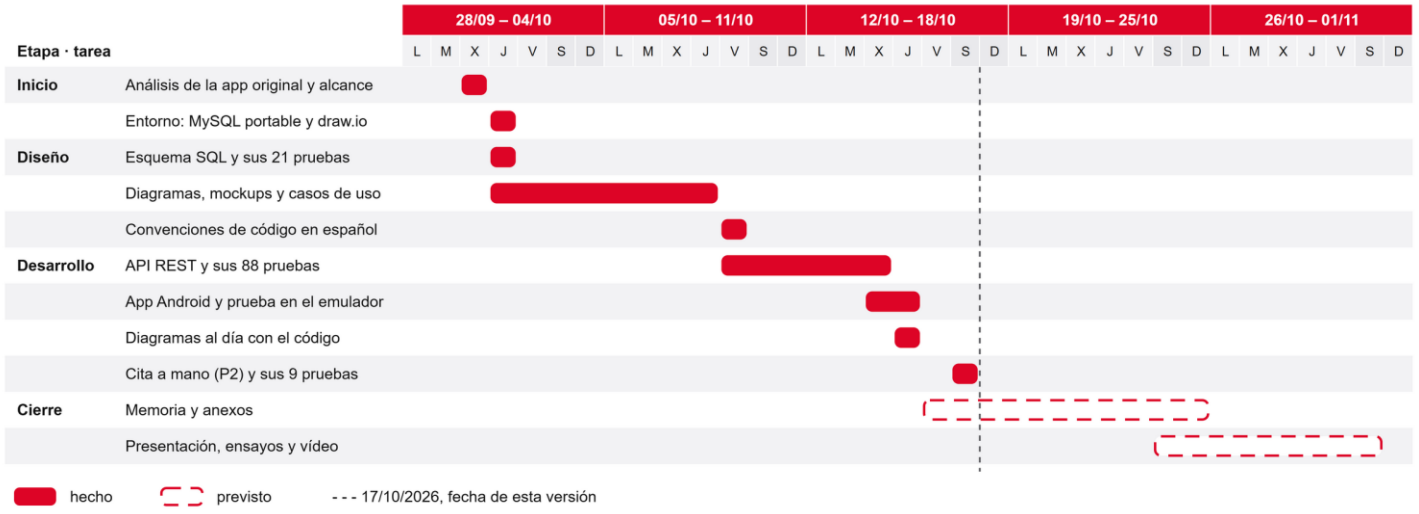
Ciclo incremental · 78 horas estimadas junto al trabajo

Inicio
30 sep – 1 oct

Diseño
1 – 9 oct

Desarrollo
9 – 17 oct

Cierre
16 – 31 oct



5. Desarrollo

Arquitectura: la app nunca toca la base de datos

APP ANDROID

- Java 17 y vistas XML
- Material 3
- MVVM con LiveData
- Red en un hilo aparte
- Sesión cifrada con el Keystore

HTTP + JSON

token JWT

API REST

- Java 21 y HttpServer del JDK
- Rutas y JSON escritos a mano
- 24 rutas y 8 hilos
- Capas http, seguridad, servicio y repositorio

JDBC

PreparedStatement

MYSQL 9.7

- 5 tablas
- Índice único contra dobles reservas
- Usuario de la API sin permisos sobre las tablas
- Escucha solo en 127.0.0.1

Solo la API habla con MySQL, así las credenciales de la base nunca viajan dentro del APK.

5. Desarrollo

Seguridad revisada con OWASP MASVS

Contraseñas

PBKDF2-HMAC-SHA256 con 600.000 iteraciones y una sal de 16 bytes

Token de sesión

JWT HS256 firmado a mano con una clave de 32 bytes o más, válido 7 días

Sesión en el móvil

Cifrada con AES-GCM y una clave que vive en el Android Keystore

Intentos de acceso

Tras 5 fallos en 15 minutos la API responde 429

Consultas

Siempre con PreparedStatement, nunca SQL concatenado

Red y copias

HTTP en claro solo hacia el emulador en debug, y copias de seguridad de la app desactivadas

5. Desarrollo

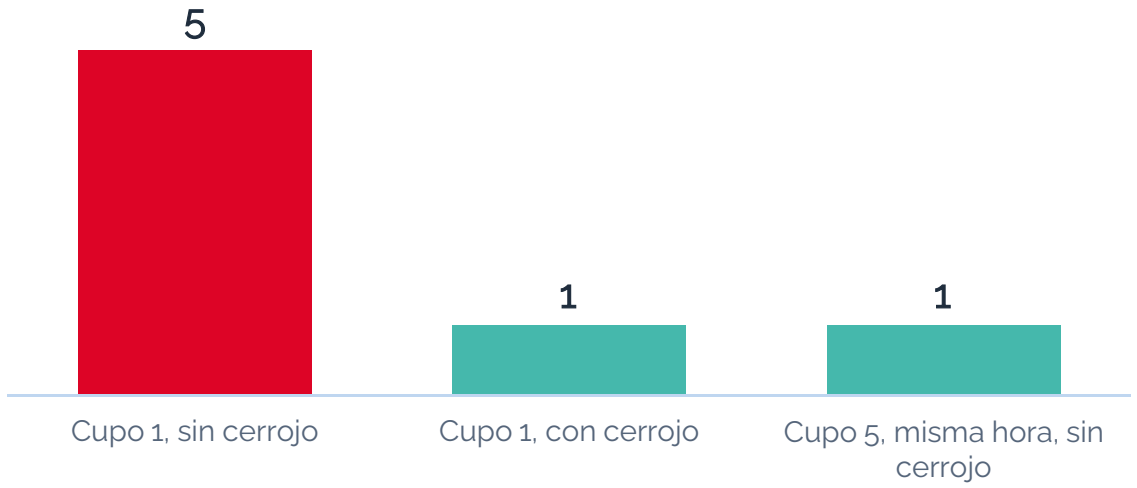
Concurrencia: dos clientes y una sola plaza libre

1. Cada petición cuenta las citas del día y ve 0 de 1
2. Las dos creen que queda sitio e insertan su cita
3. El día acaba con dos citas cuando el cupo era una

LA SOLUCIÓN

Un cerrojo por fecha con synchronized. Contar e insertar van dentro de la misma transacción, y el índice único queda como última defensa.

Citas creadas con 5 peticiones a la vez



Prueba automática: 5 hilos que salen a la vez con un CountDownLatch y un repositorio que tarda 300 ms en contar.

5. Desarrollo

Pruebas: 144 pruebas automáticas en verde

97

pruebas de la API

11 clases de JUnit 6 con una base de datos aparte y un reloj que se puede mover

26

pruebas de la app

Modelos, calendario, lista de horas, formatos y validaciones del registro

21

pruebas del esquema

Restricciones, índice único y datos de ejemplo en MySQL

Recorrido en el emulador

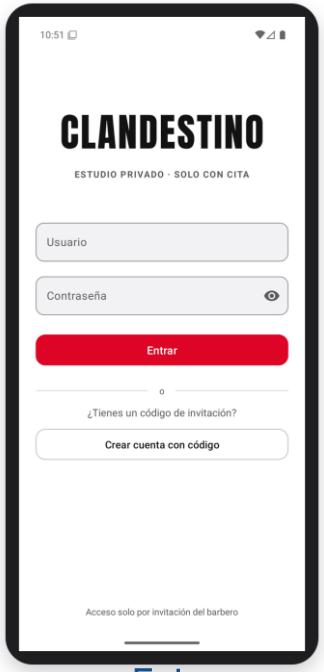
10 casos, del registro con código a la cita que el barbero mete a mano, todos correctos

Comprobaciones con curl

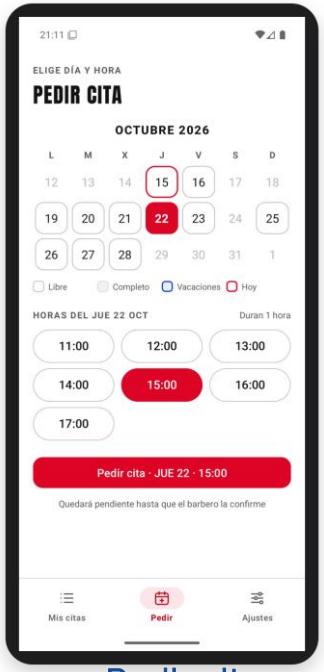
8 respuestas de la API revisadas a mano: 200, 401, 404, 405 y 400 con su código de error

5. Desarrollo

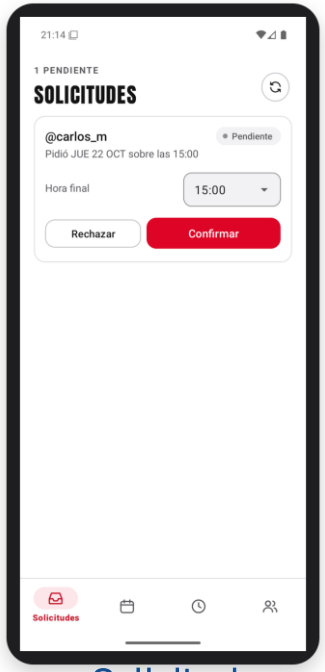
Demostración: la app funcionando en el emulador



1 · Entrar



2 · Pedir cita



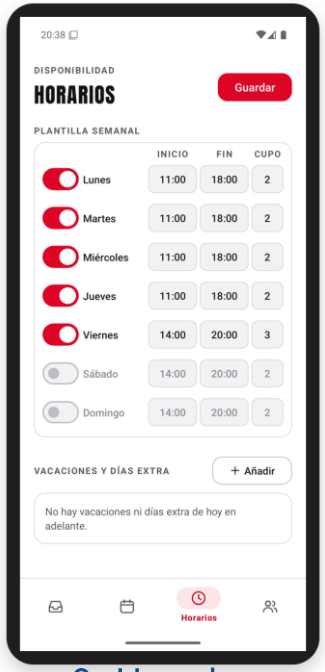
3 · Solicitudes



4 · Agenda



5 · Cita a mano



6 · Horarios

6. Conclusiones y proyección a futuro

Objetivos cumplidos y lo que falta

5 / 5

objetivos cumplidos

- Prototipo P1 completo, más la cita a mano de la P2
- Los 30 requisitos funcionales programados
- Recorrido completo documentado con capturas

Limitaciones

- Sin HTTPS, solo apto para uso local
- El cerrojo vive en la memoria de una única API
- Las solicitudes pendientes no caducan solas
- Sin pruebas de interfaz ni copia automática

Trabajo futuro

- Resto de la P2, como la caducidad de las solicitudes
- Avisos al cliente
- Resumen mensual de ingresos
- API publicada con HTTPS

AGRADECIMIENTOS

GRACIAS POR SU ATENCIÓN

El código, la memoria y los anexos de instalación están en github.com/Jhonnatanospina/clandestino-tfg

